# 1. Mục tiêu

- Hiểu rõ tư tưởng và sự khác biệt giữa Clean Architecture và Layered Architecture.
- Nắm được các nguyên tắc cơ bản để thiết kế cấu trúc thư mục và phân chia trách nhiệm trong một dự án phần mềm.
- Có khả năng áp dụng một trong hai kiến trúc để tổ chức code cho dự án của mình.
- Chuẩn bị tốt cho phần thuyết trình và demo cấu trúc code.

# 2. Nội dung cần nghiên cứu và chuẩn bị

## 2.1. Layered Architecture (Kiến trúc phân lớp)

- **Khái niệm:** Là một kiến trúc phổ biến, chia ứng dụng thành các lớp (layers) theo chiều ngang, mỗi lớp có một trách nhiệm cụ thể và chỉ giao tiếp với lớp bên dưới nó. Các lớp điển hình: Presentation (UI), Business Logic (Service), Data Access (Repository/DAL).
- **Ưu điểm:** Dễ hiểu, dễ triển khai, phân tách rõ ràng trách nhiệm.
- **Nhược điểm:** Sự phụ thuộc chặt chẽ giữa các lớp, khó thay đổi nếu có sự thay đổi lớn ở lớp dưới.
- **Ví dụ:** Minh họa cấu trúc thư mục và luồng dữ liệu trong một ứng dụng web sử dụng kiến trúc N lớp tuỳ theo

## 2.2. Clean Architecture (Kiến trúc sạch)

- **Khái niệm:** Là một tư tưởng thiết kế kiến trúc phần mềm tập trung vào việc tách biệt các mối quan tâm (separation of concerns) và đảm bảo tính độc lập của các thành phần cốt lõi (Domain Entities, Use Cases) khỏi các chi tiết triển khai (UI, Database, Framework). Nó được xây dựng dựa trên các nguyên lý SOLID.
- **Các vòng tròn (Layers):** Entities, Use Cases (Interactors), Interface Adapters (Controllers, Presenters, Gateways), Frameworks & Drivers (Web, DB, Devices).
- **Quy tắc phụ thuộc (Dependency Rule):** Các mối quan hệ phụ thuộc chỉ được phép hướng vào trong (từ vòng ngoài vào vòng trong). Vòng trong không được biết gì về vòng ngoài.
- **Ưu điểm:** Độc lập với Framework, UI, Database; dễ test, dễ bảo trì, dễ mở rộng.
- **Nhược điểm:** Phức tạp hơn trong giai đoạn đầu, yêu cầu hiểu biết sâu về thiết kế.
- **Ví dụ:** Minh họa cấu trúc thư mục và luồng dữ liệu trong một ứng dụng theo Clean Architecture. Tập trung vào việc làm thế nào để Domain và Use Cases không phụ thuộc vào Web hay Database. Cần một bài toán có nghiệp vụ đủ rộng để demo về phần này

## 2.3. So sánh và lựa chọn

- **So sánh:** Phân tích điểm giống và khác nhau giữa Layered Architecture và Clean Architecture. Khi nào nên sử dụng kiến trúc nào?
- **Lựa chọn:** Sinh viên có thể tự chọn số tầng tùy ý (ví dụ: Controller, Service, Infrastructure...) hoặc áp dụng các vòng tròn của Clean Architecture. Quan trọng là phải giải thích được lý do lựa chọn và cách tổ chức code.

## 2.4. Repository Pattern và Entity Framework Core

- **Repository Pattern:** Một pattern giúp tách biệt logic truy cập dữ liệu khỏi Business Logic. Nó cung cấp một tập hợp các phương thức để thao tác với dữ liệu (CRUD).

- **Entity Framework Core (cho ASP.NET Core):** Là một ORM (Object-Relational Mapper) giúp lập trình viên làm việc với database bằng cách sử dụng các đối tượng .NET. EF Core đã có sẵn các cơ chế tương tự Repository Pattern (ví dụ: `DbContext`, `DbSet`).
- **Lưu ý:** Nếu dùng ASP.NET Core, có thể tận dụng các tính năng của EF Core thay vì tự implement Repository Pattern từ đầu.

## 3. Chuẩn bị thuyết trình và demo

- **Slide:** Trình bày rõ ràng về Layered Architecture và Clean Architecture, tập trung vào các nguyên tắc, ưu nhược điểm và cách áp dụng.
- **Code Demo:**
  - Demo cấu trúc thư mục của một dự án theo kiến trúc đã chọn.
  - Trình bày luồng xử lý của một chức năng đơn giản (ví dụ: thêm/sửa/xóa một đối tượng) để minh họa cách các lớp/vòng tròn tương tác với nhau.
  - Giải thích cách các nguyên lý SOLID được áp dụng trong kiến trúc này.
- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo.

## 4. Phân công công việc và ước lượng thời gian (Gợi ý)

| Task | Người thực hiện (Gợi ý) | Ước lượng (Point) |
|---|---|---|
| Nghiên cứu sâu về Layered Architecture | TV1 | 2 |
| Nghiên cứu sâu về Clean Architecture | TV2 | 3 |
| So sánh hai kiến trúc và lựa chọn | Nhóm trưởng | 2 |
| Thiết kế cấu trúc thư mục cho dự án demo | Cả nhóm | 3 |
| Viết code demo một luồng chức năng đơn giản | TV3, TV1 | 4 |
| Chuẩn bị slide thuyết trình | TV2, Nhóm trưởng | 3 |
| Tổng hợp, kiểm tra và chạy thử demo | Cả nhóm | 2 |
| Luyện tập thuyết trình và demo | Cả nhóm | 1 |
| **Tổng cộng** | | **20** |

## 5. Tài liệu tham khảo

- **Sách:** "Clean Architecture" của Robert C. Martin (Uncle Bob).
- **Bài viết:** Tìm kiếm các bài viết về Layered Architecture, Clean Architecture, Repository Pattern.
- **Tài liệu chính thức:** Microsoft Docs về kiến trúc ASP.NET Core, Entity Framework Core.

## 6. Ghi chú quan trọng

- **Tư tưởng quan trọng hơn Pattern:** Clean Architecture là một tư tưởng, không phải một pattern cứng nhắc. Hãy hiểu nguyên tắc và áp dụng linh hoạt.
- **Bắt đầu đơn giản:** Không cần cố gắng áp dụng tất cả các khái niệm ngay từ đầu. Bắt đầu với Layered Architecture và dần dần nâng cấp lên Clean Architecture khi đã hiểu rõ.

- **Giải thích rõ ràng:** Khi demo, hãy giải thích rõ lý do tại sao nhóm lại tổ chức code như vậy và nó mang lại lợi ích gì.

---

**Chúc các bạn học tập hiệu quả và thành công!**